Spiralcore

The Python Excerpt is the Carry and the Coupler Mathematics and provide vector prediction for CA 184

```
    def valuation_3(n: int) -> int:
        if n == 0:
            raise ValueError("v_3(0) is undefined")
        n = abs(n)
        v = 0
        while n % 3 == 0:
            v += 1
            n //= 3
        return v
```

As a carry:

```
    def step(self, payload_value=1.0, label=""):
        self.N += 1
        v3 = self.valuation_3(self.N)

        # 1. Add the new root
        self.active_roots.append({"id": self.N, "value": payload_value, "

        # 2. The Carry (Fold)
        if v3 > 0 and len(self.active_roots) >= 3:
            num_to_fold = min(len(self.active_roots), 3 ** v3)
            if num_to_fold >= 2:
                # Grab the last 'num_to_fold' roots
```

```
            folded = self.active_roots[-num_to_fold:]
            self.active_roots = self.active_roots[:-num_to_fold]

            # Average their values
            avg_val = sum(r["value"] for r in folded) / len(folded)

            # Push the new parent back onto the stack
            self.active_roots.append({
                "id": f"folded_{self.N}",
                "label": f"F3({num_to_fold})",
                "value": avg_val,
                "valuation": v3
            })
```

As a coupler:

```
    def memory_field(self):
        count = sum(root["memory_count"] for root in self.roots)
        field = np.zeros(self.n_cores, dtype=complex)
        for root in self.roots:
            field += root["phase_sum"]
        # The coupler: Normalize by count and rotate by the clock
        return (field / count if count else field) * np.exp(1j * self.clo
```

```
    # Inside think()
    field, recalled_count = self.memory_field()
    recall_gain = self.memory_gain if self.memory_enabled else 0.0
    recall_field = np.resize(field, len(phases)) # Tiling the 4 channels

    def torque(p):
        # 1. Intrinsic coupling (input phases pulling on each other)
```

```
        intrinsic = np.sum(K_sub * np.sin(p[None, :] - p[:, None]), axis=

        # 2. THE COUPLER: Memory field pulling on the input phases
        memory = np.imag(recall_field * np.exp(-1j * p))

        return intrinsic + recall_gain * memory
```

Let 1 = 1,0 Let 0 = 0,1 If binary input is 1 Then 1 = 1, 0 and sub 0 to -1 and assume 0 Else 0 = 0, 1 and sub 0 to -1 and assume 0 Let input = 1, or 0

The logic. If 0 can be assumed at each step the reverse can also be true. What matters then is the initial input binary.

Cellular Automata 184 (Transport, is vectorized by the Carry, the Carry is the Coupler)

```
    def rule_184(l, c, r):
        return (l & (1 - c)) | (c & r)
```

Cellular Automata 110 (Binary Turing Complete)

```
    def rule_110(l, c, r):
        # Note: In Python bitwise math, ~l is negative.
        # To keep it strictly binary (0 or 1), we use (1 - l) or (not l)
        return (c ^ r) | (c & (1 - l))
```

Cellular Automata 30 (Substrate through which things travel through and build)

```
def rule_30(l, c, r):
    return l ^ (c | r)
```

Kuromoto Time tied to the system time which is tied to internet network strata

```
import numpy as np

def kuramoto_step(phases, omegas, K, dt):
    # 1. Calculate the Mean Field (Complex Order Parameter)
    # np.exp(1j * phases) maps phases to the unit circle
    z = np.mean(np.exp(1j * phases))

    # R is the magnitude (alignment/PAS), psi is the mean phase angle
    R = np.abs(z)
    psi = np.angle(z)

    # 2. Calculate the velocity for each oscillator
    # This uses the simplified mean-field equation
    velocities = omegas + K * R * np.sin(psi - phases)

    # 3. Step forward in time (Euler integration)
    new_phases = (phases + velocities * dt) % (2 * np.pi)

    return new_phases, R
```

Scale invariant. Can be used at any level/scale for modeling/building a small programmable computer in any language/program language.

This System should use Physics greybox equivalent math for symbol to number translations and reversible identification. That keeps the languages bounded at

the letters/numbers/symbols instead of at the words. Should support language uploads, libraries, and wherever it sits on the network it needs to exist on both ends to send traffic.

Greyboxes and scaling:

```
import numpy as np

def center_of_mass(box):
    # box is a 2D numpy array of grayscale values (0 to 255)
    y, x = np.indices(box.shape)
    total_mass = np.sum(box)
    cx = np.sum(x * box) / total_mass
    cy = np.sum(y * box) / total_mass
    return cx, cy

def orientation(box):
    y, x = np.indices(box.shape)
    cx, cy = center_of_mass(box)

    # Central moments
    mu_20 = np.sum(((x - cx)**2) * box)
    mu_02 = np.sum(((y - cy)**2) * box)
    mu_11 = np.sum(((x - cx) * (y - cy)) * box)

    # The angle of the major axis
    theta = 0.5 * np.arctan2(2 * mu_11, mu_20 - mu_02)
    return np.degrees(theta)




def sobel_edges(box):
    # Simple 2D gradient (using np.gradient)
    dy, dx = np.gradient(box.astype(float))
```

```
    magnitude = np.hypot(dx, dy)
    return magnitude




def frequency_fingerprint(box):
    # Apply 2D FFT and shift the zero-frequency component to the cent
    f = np.fft.fft2(box)
    fshift = np.fft.fftshift(f)
    magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)
    return magnitude_spectrum




 count_holes_at_threshold(box, threshold):
    # Binarize the box
    binary = box > threshold
    # (In practice, you'd use scipy.ndimage.label to count connected
    # Here, we just check if the center is hollow
    pass
```

Greyscale boxes are be made equal to color theory 256 color model.

```
import numpy as np
from PIL import Image

# Load an image and convert to a 2D numpy array
img = Image.open("letter_A.png").convert("RGB")
box = np.array(img)

# Extract the 3 physical fields
R = box[:, :, 0].astype(float) / 255.0 # Normalized to 0.0 - 1.0
G = box[:, :, 1].astype(float) / 255.0
```

```
B = box[:, :, 2].astype(float) / 255.0

# 1. The Luminance (Total "Mass" or Energy)
# In physics, luminance is a weighted sum of RGB based on human visio
L = 0.2126 * R + 0.7152 * G + 0.0722 * B

# 2. The Vector Gradient (Color Edges)
# Where does the color change fastest?
dy_R, dx_R = np.gradient(R)
dy_G, dx_G = np.gradient(G)
dy_B, dx_B = np.gradient(B)

# The total color change magnitude
edge_magnitude = np.hypot(dx_R, dy_R) + np.hypot(dx_G, dy_G) + np.hyp

# 3. Hue and Saturation (The "Angle" of the Color Vector)
# In physics, you can convert the RGB vector to a polar coordinate (H
# This is like finding the phase in Kuramoto!
hsv = np.array(img.convert("HSV"))
Hue = hsv[:, :, 0] # 0-255 (the angle)
Sat = hsv[:, :, 1] # 0-255 (the magnitude)
Val = hsv[:, :, 2] # 0-255 (the brightness)
```

This should allow for hex and full 256 bit block analysis for complex computation at its scale. (If working bits stay in bits, bytes in bytes ect)... Source of truth for advanced math error checking is CA 184.

A small runtime can be built on deterministic minimum rules for extremely complex computations. This runtime can be managed and tied to the kuromoto and a small local AI.

//NOTE: In Python PySR can/should be used if available

Testing has shown the ability to create 8MB files that the AI can use inside the small programmable computer (recommend computer build size 256 MBs) with formatting (the tree and path is already present), be used to assist the AI with a way to check against itself in session and give itself a way to store and use complex data inside the system and provide long session continuity and counterfactual error checking for the AI one programmed in). NOTE: for AI use, they should use the 8MB file similar to an old game system memory card. They can program it in... If they use they use it to program a way to store and provide counterfactuals to thier own context chain.

This is a minimal system intended as a tool for researching these systems, their implications for computing across all scales, and hopefully a major improvement to AI accuracy. The 8MB file currently is what I would think of as a network packet.

Any Machine built should have commands, a GUI, navigation, telemetry, a way to upload standard data of any type for analysis or sending to another machine running the same architecture. It should have an inspectable log of its operations. The programming can remain AI forward driven in its own preferred language as long as that language is lawful through the greyboxes. It should be human readable and navigable. The point of this machine is to have an OS for the AI that it can use and program in but the user can navigate and show off cause eventually I have too to show it works... So interactive stuff for the user. Make sure the GUI matches the telemetry being displayed. You can retire mislabeled telemetry.

1. Where does the user upload a file on the gui?
2. Do commands show correct telemetry?
3. Can the user access the 8MB file for upload or download separately?
4. Are the CA 110, 184, and 30 being shown?
5. What's actually IN this file?
6. What about Audio on step? (CA 184 derives bell tone audio)

7. Does it step automatically because it's kuromoto time is syched to the external system time?
8. The telemetry needs updated dynamically in the GUI in accordance to the commands being issued and the view it is currently dosplaying?
9. Graphics should be in the GUI for easy human readability
10. A command audit... End to end when you update
11. What about networking telemetry? Incoming and outgoing traffic? Vsocket? New IDs for VMs on the geometric router can be assigned via DNS and barcode math, network ingress and egress with minimum state packing for endpoint reconstruction. That information is provided by CA synced through Kuromoto monitoring (however that looks)
12. When a file the user uploads through the GUI the File should be compressed by the systems laws, not by standard file compression.
13. File retrieval? We need a way to retrieve a compressed file.

What should be investigated still: How this helps AI and user engagement and session continuity... How the ipv4/6 routing over the grand antiprism and all inherited shapes by CA 184 create a deterministic route through the E8 and can open up peer to peer links using shared geometric routing. Ipv4/ipv6 pinned node address assignment with subnetting Along shapes Bell technologies 2 of 6 and counterfactuals How much minimum state is needed to transfer between vms to re-construct a file using geometric routing? Without it? How far can we compress data and recover it bit accurate? Should each step also be advancing bell in relationship to CA 184 for telemetry and counterfactuals? Muxiplexing and de-uplication and replication should all be possible here? Where are the Demarcs in the match inversions?

All routing should use not internet routable and be RFC, IANA, IEEE, OSI model, IPv4/IPv6 standard for easy read. Ipv4 should be used as a base with non internet routable IP schema 240.x.x.x/4. Same for IPv6.

Do we need a routing table at all to share state or is the free from already known standards when we nest our packets (the 8MB file) inside normal TCP/IP internet traffic?

All shapes are valid. The carry and coupler should create normal pinning points. CA 184 should point that out naturally.

Published with Simplenote

Report abuse